

OpenFlex 开源轮臂人形机器人使用说明书（高级篇）

English | 中文

本文档为 OpenFlex 机器人系统的高级使用说明，覆盖传感器系统、建图、定位、导航、全身 VLA 大模型数据采集/训练/推理，以及核心算法技术细节与论文依据。基于 ROS 2 Humble 平台，整机由四轮独立转向全向底盘 + 升降台 + 双 7 自由度机械臂 + 2 自由度头部 + Livox Mid-360 SLAM 导航 + VR 遥操作 + LeRobot VLA 等子系统组成。

目录

- 第一章 传感器系统
 - 1.1 传感器快速开始
 - 1.2 Livox Mid-360 激光雷达
 - 1.3 IMU 惯性测量单元
 - 1.4 其他传感器
- 第二章 建图
 - 2.1 建图快速开始
 - 2.2 FAST-LIO2 算法原理
 - 2.3 建图模式配置
 - 2.4 PGO 位姿图优化
 - 2.5 3D 点云转 2D 栅格地图
- 第三章 定位
 - 3.1 定位快速开始
 - 3.2 导航模式下的 LIO 与 LIVO 里程计
 - 3.3 ICP 点云配准定位
 - 3.4 ScanContext 回环检测与重定位
 - 3.5 ICP 三级定位策略
 - 3.6 ICP 持续重对齐机制
 - 3.7 TF 树与坐标系关系
 - 3.8 定位精度与漂移分析
- 第四章 导航
 - 4.1 导航快速开始
 - 4.2 Nav2 整体架构
 - 4.3 NMPC 局部控制器
 - 4.4 全局路径规划器
 - 4.5 代价地图配置
 - 4.6 速度平滑器
 - 4.7 行为服务器与恢复行为
 - 4.8 3D 点云转 2D 激光扫描（实时）
- 第五章 全身VLA大模型VR数据采集、训练与推理
 - 5.1 全身 VLA 快速开始
 - 5.2 LeRobot 接入与全身采集目标
 - 5.3 OpenFlex 全身数据 Schema
 - 5.4 底盘观测与动作参考对比
 - 5.5 Follower（Robot）实现
 - 5.6 Teleoperator（Leader）实现
 - 5.7 全身数据录制流程
 - 5.8 数据质量检查与标定要求
 - 5.9 训练流程（lerobot-train）
 - 5.10 推理与部署
 - 5.11 长程任务分层与渐进启用
- 附录 核心算法技术细节与论文依据
 - A.1 算法索引与项目实现位置
 - A.2 四转四驱舵轮运动学与里程计
 - A.3 FAST-LIO2 与 ikd-Tree
 - A.4 FAST-LIVO2 视觉-激光-惯性融合
 - A.5 ScanContext、NDT、ICP 与重定位
 - A.6 PGO、iSAM2 与 HBA 地图精修
 - A.7 3D 点云到 Nav2 2D 栅格/扫描
 - A.8 Nav2 SmacPlanner2D、局部 A* 与 NMPC
 - A.9 VR 遥操作、双臂 IK 与安全限幅
 - A.10 CANopen/CiA402 升降台控制与回零
 - A.11 LeRobot VLA 策略、ACT、Diffusion Policy 与 VLA 模型
 - A.12 参考论文与标准清单

第一章 传感器系统

1.1 传感器快速开始

本节用于快速检查 Livox Mid-360 点云和 IMU 是否正常。建图、定位、导航流程通常由 `swerve_navigation` 的 `launch` 自动启动 Mid-360 驱动；只有在单独调试传感器时，才需要直接启动 `livox_ros_driver2`。

1.1.1 确认网络配置

MID360 通过以太网输出点云和 IMU 数据。当前项目默认配置如下：

项目	值
主机 IP	192.168.1.50
子网掩码	255.255.255.0
MID360 IP	192.168.1.173

在 Ubuntu 图形界面中配置：

- 1. 打开 设置 -> 网络。
- 2. 选择连接 MID360 的有线网卡，例如 `eno1`。
- 3. 点击齿轮图标，进入 `IPv4` 页面。
- 4. 将 IPv4 方式改为 手动。
- 5. 添加地址 `192.168.1.50`，子网掩码 `255.255.255.0`，网关一般留空。
- 6. 点击 应用 后，重新插拔网线，或关闭再打开该有线连接。

也可以用项目脚本临时配置并检查连通性：

```
cd ~/openflex_all/openflex_ws
./scripts/setup_mid360_network.sh
```

等价的手工命令：

```
sudo ip addr add 192.168.1.50/24 dev eno1
sudo ip link set eno1 up
```

检查主机是否能连通 MID360：

```
ping 192.168.1.173
```

1.1.2 单独启动 Mid-360 驱动

启动前加载工作空间：

```
cd ~/openflex_all/openflex_ws
source install/setup.bash
```

不启动 RViz，仅启动 MID360 驱动：

```
ros2 launch livox_ros_driver2 msg_MID360_launch.py
```

该方式默认发布：

话题	类型	说明
/livox/lidar	livox_interfaces2/msg/CustomMsg	激光点云
/livox/imu	sensor_msgs/msg/Imu	MID360 内置 IMU

1.1.3 检查传感器数据

```
ros2 topic list | grep livox
ros2 topic info /livox/lidar
ros2 topic info /livox/imu
ros2 topic hz /livox/lidar
ros2 topic hz /livox/imu
```

基本判断：

检查项	正常现象
ping 192.168.1.173	能收到 MID360 响应
/livox/lidar	有话题，频率通常约 10 Hz
/livox/imu	有话题，频率通常约 200 Hz
RViz 点云	转动或移动雷达时，点云能实时变化

1.1.4 可视化查看点云

```
ros2 launch livox_ros_driver2 rviz_MID360_launch.py
```

该方式默认把 /livox/lidar 发布为标准 ROS 点云 sensor_msgs/msg/PointCloud2，便于 RViz 直接显示。

1.1.5 与建图/导航 launch 的关系

swerve_navigation 的建图和导航 launch 会启动 Mid-360 驱动，一般不需要先手动运行。如果已经手动启动了 Livox 驱动，再启动建图或导航 launch，可能出现重复节点或端口占用。推荐流程是：先用本节命令确认传感器正常，停止独立驱动后，再进入建图、定位或导航流程。

常见问题：

现象	优先检查
ping 不通	网线、雷达供电、主机 IP、网卡名
有 /livox/lidar 但无频率	MID360 IP、MID360_config.json、防火墙/网络
/livox/imu 无数据	驱动是否正常启动，配置文件是否匹配
RViz 没有点云	Fixed Frame、话题类型、是否使用了带 RViz 的 launch

1.2 Livox Mid-360 激光雷达

1.2.1 传感器规格

参数	值
测量距离	0.3 ~ 40 m
FOV	360° × 59°（水平 × 垂直）
点频	~200,000 pts/s
扫描模式	非重复扫描（随时间覆盖更大 FOV）
内置 IMU	6 轴，200 Hz
输出帧率	10 Hz（可配置）
数据接口	以太网
ROS 消息类型	livox_ros_driver2/msg/CustomMsg

1.2.2 ROS 驱动配置与启动

代码文件位置： src/openflex_chassis/hardware_sensor_layer/livox_ros_driver2/

网络配置文件： src/openflex_chassis/hardware_sensor_layer/livox_ros_driver2/config/MID360_config.json

关键网络配置：

```
"host_net_info": {
  "cmd_data_ip": "192.168.1.50",
  "point_data_ip": "192.168.1.50",
  "imu_data_ip": "192.168.1.50"
}

"lidar_configs": [
  {
    "ip": "192.168.1.173"
  }
]
```

默认驱动参数：

参数	默认值	说明
xfer_format	1	发布 Livox 自定义点云消息
multi_topic	0	所有雷达共用一个点云话题和一个 IMU 话题
data_src	0	数据源，0 表示雷达
publish_freq	10.0	点云发布频率
output_data_type	0	输出数据类型
frame_id	livox_frame	消息坐标系

1.2.3 发布话题

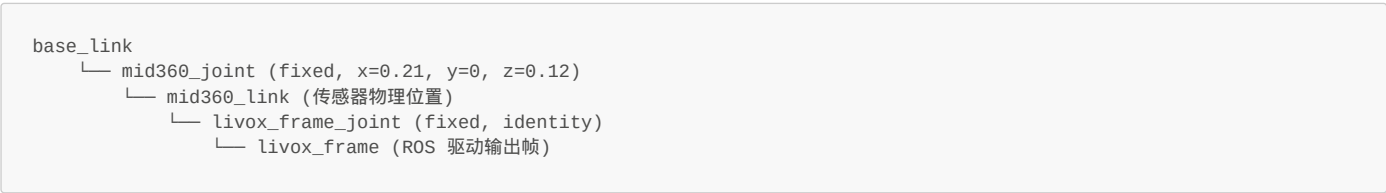
默认启动方式 (msg_MID360_launch.py)：

话题	类型	说明
/livox/lidar	livox_interfaces2/msg/CustomMsg	激光点云
/livox/imu	sensor_msgs/msg/Imu	MID360 内置 IMU

RViz 启动方式 (rviz_MID360_launch.py)：

话题	类型	说明
/livox/lidar	sensor_msgs/msg/PointCloud2	标准 ROS 点云
/livox/imu	sensor_msgs/msg/Imu	MID360 内置 IMU

1.2.4 坐标系关系



livox_frame 和 mid360_link 在相同位置，livox_frame 是为了兼容 ROS 驱动的帧名要求。

1.3 IMU 惯性测量单元

OpenFlex 第一版导航链路主要使用 Livox Mid-360 内置 IMU。IMU 数据由 livox_ros_driver2 随点云驱动一起发布，不需要单独启动 IMU 节点。

项目	说明
话题	/livox/imu
类型	sensor_msgs/msg/Imu
典型频率	约 200 Hz
使用方	FAST-LIO2 / FAST-LIVO2 建图、定位与里程计估计

检查 IMU 是否正常：

```
ros2 topic info /livox/imu
ros2 topic hz /livox/imu
ros2 topic echo --once /livox/imu
```

如果 /livox/lidar 正常但 /livox/imu 没有数据，优先检查 MID360 驱动是否使用了正确的 MID360_config.json，以及雷达固件、供电和网络连接是否稳定。

1.4 其他传感器

当前底盘 URDF 中保留了 D435 深度相机和 RPLiDAR C1 的安装位，用于扩展、调试或外部方案对接。第一版建图、定位、导航主链路默认依赖 Livox Mid-360 点云和内置 IMU，不要求额外启动这些传感器。

传感器	默认角色	说明
Intel D435	可选视觉/深度输入	建图 launch 中可通过 use_camera:=true 启动，用于视觉融合或调试

传感器	默认角色	说明
RPLiDAR C1	备用 2D 激光雷达	URDF 中保留安装位，默认导航链路使用 Mid-360 点云投影出的 <code>/scan</code>

如需启用 D435：

```
ros2 launch swerve_navigation mapping.launch.py use_camera:=true
```

第二章 建图

2.1 建图快速开始

`swerve_navigation` 是 OpenFlex 建图、定位、导航的推荐入口。建图时建议从 `ros2 launch swerve_navigation mapping.launch.py` 启动完整链路，不建议直接把 `pgo`、`fast_livo` 等底层包作为主入口。

以下示例统一使用地图名 `your_map`：

```
export MAP_NAME=your_map
```

2.1.1 建图准备

建图前可先单独检查底盘运动是否正常：

```
ros2 launch swerve_bringup swerve_drive.launch.py
```

另开终端启动键盘控制：

```
ros2 run swerve_bringup swerve_teleop.py
```

检查完成后停止 `swerve_drive.launch.py`，避免和 `mapping.launch.py` 重复启动底盘控制链路。

创建 3D 地图目录：

```
mkdir -p "$HOME/MID_360_nav/openflex_maps/3d/$MAP_NAME"
```

2.1.2 启动建图节点

```
ros2 launch swerve_navigation mapping.launch.py
```

常用可选参数：

参数	默认值	含义
<code>use_rviz</code>	<code>true</code>	是否启动 RViz
<code>use_camera</code>	<code>false</code>	是否启动 D435 相机
<code>steering_can_interface</code>	<code>can5</code>	转向电机 CAN 接口
<code>driving_can_interface</code>	<code>can4</code>	驱动电机 CAN 接口
<code>mapping_max_wheel_speed</code>	<code>2.0</code>	建图时轮速上限
<code>mapping_wheel_accel_limit</code>	<code>1.2</code>	建图时轮速变化率上限

实时建图时，RViz 中主要关注 `/pgo/global_cloud` 和 `/fastlio2/body_cloud`。

2.1.3 保存 3D 地图

建图完成后调用 PGO 保存服务：

```
mkdir -p "$HOME/MID_360_nav/openflex_maps/3d/$MAP_NAME"

ros2 service call /pgo/save_maps interface/srv/SaveMaps \
  "{file_path: '$HOME/MID_360_nav/openflex_maps/3d/$MAP_NAME', save_patches: true}"
```

成功后会生成：

```
~/MID_360_nav/openflex_maps/3d/your_map/
├── map.pcd
├── poses.txt
├── patches/
└── sc_data/
```

2.1.4 查看 3D 点云地图

```
ros2 launch pgo view_saved_map.launch.py \
  pcd_path:="$HOME/MID_360_nav/openflex_maps/3d/$MAP_NAME/map.pcd"
```

2.1.5 3D 地图转 2D 导航地图

```
mkdir -p "$HOME/MID_360_nav/openflex_maps/2d/$MAP_NAME"

ros2 run pgo pcd_to_nav2_map \
  --pcd "$HOME/MID_360_nav/openflex_maps/3d/$MAP_NAME/map.pcd" \
  --out-dir "$HOME/MID_360_nav/openflex_maps/2d/$MAP_NAME" \
  --poses "$HOME/MID_360_nav/openflex_maps/3d/$MAP_NAME/poses.txt" \
  --resolution 0.05 \
  --z-min 0.00 \
  --z-max 2.30 \
  --obstacle-z-min 0.10 \
  --obstacle-z-max 1.40 \
  --inflation-radius 0.10
```

输出：

```
~/MID_360_nav/openflex_maps/2d/your_map/
├── map.yaml
└── map.pgm
```

2.1.6 可选修图

可以使用图形界面生成和修图：

```
ros2 run pgo pcd_to_nav2_map_gui
```

2.1.7 2D 地图效果调试

现象	建议调整
地面被当成障碍	提高 <code>--obstacle-z-min</code> ，例如从 <code>0.10</code> 调到 <code>0.15</code>
高处结构投影成障碍	降低 <code>--obstacle-z-max</code>
地图噪点多	增加 <code>--sor-mean-k 20 --sor-stddev 1.0</code>
地图太细碎	将 <code>--resolution</code> 放宽到 <code>0.10</code>
障碍物边界太贴近机器人	增大 <code>--inflation-radius</code>

2.2 FAST-LIO2 算法原理

2.2.1 算法概述

FAST-LIO2（Fast LiDAR-Inertial Odometry 2）是一种紧耦合的激光-惯性里程计算法，使用 **IESKF**（迭代误差状态卡尔曼滤波器）和 **IKD-Tree**（增量 KD 树）实现高效的状态估计和地图维护。

代码文件位置：`src/openflex_chassis/mapping_localization_layer/fastlio2/src/lio_node.cpp`

2.2.2 IESKF 状态估计

状态向量包含：

- 位置 `t_wi`：世界坐标系下 IMU 的位置
- 速度 `v`：世界坐标系下的线速度
- 旋转 `r_wi`：世界到 IMU 的旋转矩阵
- IMU 偏置：加速度计偏置 `ba`、陀螺仪偏置 `bg`
- 外参：LiDAR → IMU 旋转 `r_il` 和平移 `t_il`（可选在线标定）

算法流程（每帧）：

1. IMU 预积分

├ 收集两帧激光之间的所有 IMU 数据

├ 通过中值积分 (midpoint integration) 预测状态

└ 传播协方差矩阵

2. 激光点去畸变

├ 根据 IMU 预积分的连续位姿

└ 将每个点补偿到帧末时刻

3. IESKF 迭代更新（最多 `ieskf_max_iter` 次）

├ 在 IKD-Tree 中搜索每个点的最近邻 (`near_search_num` 个)

├ 计算点到面残差

├ 构建雅可比矩阵

├ 卡尔曼增益更新状态

└ 检查收敛（状态增量 < 阈值）

4. 更新 IKD-Tree

├ 将新观测点插入动态地图

└ 删除超出 `cube_len` 范围的旧点

2.2.3 IKD-Tree 动态地图

参数	值	说明
<code>cube_len</code>	200	活动地图立方体边长 (m)
<code>map_resolution</code>	0.2 m	地图体素分辨率
<code>det_range</code>	30 m	特征检测范围
<code>move_thresh</code>	0.5 m	触发地图更新的移动阈值

当机器人移动超过 `move_thresh` 时，将新扫描插入 IKD-Tree 并裁剪超出 `cube_len/2` 范围的点。

2.2.4 LiDAR-IMU 外参标定

出厂外参（Mid-360 → IMU）：

```
r_il: [1, 0, 0, 0, 1, 0, 0, 0, 1] # 旋转：单位矩阵（无旋转）
t_il: [-0.011, -0.02329, 0.04412] # 平移：LiDAR 到 IMU 的偏移 (m)
```

IMU → base_link 补偿外参（导航模式）：

```
r_ib: [1, 0, 0, 0, 1, 0, 0, 0, 1] # 旋转：单位矩阵
t_ib: [-0.21, 0.0, -0.12] # 平移：抵消 mid360_link 相对于 base_link 的偏移
```

导航模式下 `body_frame=base_link`，需要通过 `t_ib` 将输出从 IMU 位置补偿到底盘中心。建图模式下 `body_frame=mid360_link`，不需要补偿。

2.2.5 FAST-LIO2 完整参数表

配置文件位置：

- 通用基础：`src/openflex_chassis/mapping_localization_layer/fastlio2/config/lio.yaml`
- 建图模式：`src/openflex_chassis/mapping_localization_layer/fastlio2/config/lio_mapping.yaml`
- 导航模式：`src/openflex_chassis/mapping_localization_layer/fastlio2/config/lio_navigation.yaml`

参数	建图	导航	说明
<code>body_frame</code>	<code>mid360_link</code>	<code>base_link</code>	输出 TF 的子帧
<code>world_frame</code>	<code>map</code>	<code>odom</code>	输出 TF 的父帧
<code>lidar_filter_num</code>	4	3	降采样因子
<code>lidar_min_range</code>	0.5 m	0.5 m	最小有效距离

参数	建图	导航	说明
lidar_max_range	20.0 m	20.0 m	最大有效距离
scan_resolution	0.10 m	0.10 m	扫描体素大小
map_resolution	0.2 m	0.2 m	地图体素大小
cube_len	200	200	IKD-Tree 立方体边长
det_range	30 m	30 m	检测范围
move_thresh	0.5 m	0.5 m	地图更新移动阈值
na / ng	0.01 / 0.01	0.01 / 0.01	加速度计/陀螺仪噪声
nba / nbg	0.0001 / 0.0001	0.0001 / 0.0001	IMU 偏置噪声
imu_init_num	20	20	IMU 初始化采样数
near_search_num	5	5	KD 树最近邻数
ieskf_max_iter	4	3	IESKF 最大迭代次数
gravity_align	true	true	初始化时对齐重力
lidar_cov_inv	1000	1000	激光测量协方差逆
publish_map_odom	true	false	是否发布 map → odom TF

2.3 建图模式配置

2.3.1 建图模式与导航模式的核心区别

特性	建图模式	导航模式
配置文件	lio_mapping.yaml	lio_navigation.yaml
输出 TF	map → mid360_link	odom → base_link
世界帧	map	odom
体帧	mid360_link (IMU 直接输出)	base_link (补偿到底盘中心)
PGO 回环	启用 (publish_map_odom: true)	禁用
静止冻结	启用	禁用
降采样	filter_num=4 (更高质量)	filter_num=3 (更快速度)
IESKF 迭代	4 次 (更精确)	3 次 (更实时)

2.3.2 map → odom 静止冻结机制

目的：防止机器人静止时 IMU 噪声/振动导致 map → odom 变换产生微抖动。

触发条件（全部满足）：

```
1. freeze_map_odom_when_static = true (已配置)
2. map→odom 变换已经计算过至少一次
3. 没有新的 PGO 回环修正
4. 里程计消息正常接收
5. 线速度 < static_linear_thresh (0.01 m/s)
6. 角速度 < static_angular_thresh (0.02 rad/s)
7. 已静止超过 static_hold_time (0.2 s)
```

行为：冻结最后一次有效的 map → odom 变换，但仍然以 50Hz 持续发布（更新时间戳），保持 TF 树的连续性。

2.4 PGO 位姿图优化

2.4.1 算法原理

PGO 是 **Pose Graph Optimization**（位姿图优化）。它把关键帧位姿建模为图中的节点，把里程计约束、回环约束等建模为图中的边，然后通过图优化减少 FAST-LIO2 建图过程中的累计漂移，提高地图一致性。

代码文件位置：src/openflex_chassis/mapping_localization_layer/pgo/src/pgo_node.cpp

配置文件位置：src/openflex_chassis/mapping_localization_layer/pgo/config/pgo.yaml

推荐建图入口：src/openflex_chassis/navigation_layer/swerve_navigation/launch/mapping.launch.py

2.4.2 PGO 工作流程

1. 关键帧选择

├ 旋转变化 > key_pose_delta_deg (10°) 或

└ 平移变化 > key_pose_delta_trans (0.5m) 时添加关键帧

2. 回环检测

├ 在 loop_search_radius (1.0m) 内搜索候选关键帧

├ 时间间隔 > loop_time_tresh (60s)

├ 构建子地图 (±loop_submap_half_range=5 个关键帧)

├ 体素下采样到 submap_resolution (0.1m)

└ 3D 扫描匹配, 分数 < loop_score_tresh (0.15) 则接受

3. 位姿图优化

├ 将回环约束加入位姿图

└ 全局优化所有关键帧位姿

4. 发布修正

└ 通过 /pgo/offset 话题发送修正变换给 FAST-LIO2

2.4.3 PGO 参数详解

参数	值	说明
key_pose_delta_deg	10°	关键帧旋转间隔
key_pose_delta_trans	0.5 m	关键帧平移间隔
loop_search_radius	1.0 m	回环候选搜索半径
loop_time_tresh	60.0 s	回环最小时间间隔
loop_score_tresh	0.15	扫描匹配分数阈值 (越小越严格)
loop_submap_half_range	5	子地图关键帧半径
submap_resolution	0.1 m	子地图体素分辨率
min_loop_detect_duration	5.0 s	建图最短时间后才开启回环检测
map_z_min	0.1 m	保存地图时的 Z 轴下界 (去除地面)
map_z_max	3.0 m	保存地图时的 Z 轴上界 (去除天花板)

2.5 3D 点云转 2D 栅格地图

2.5.1 pcd_to_nav2_map 离线工具

代码文件位置: src/openflex_chassis/mapping_localization_layer/pgo/src/pcd_to_nav2_map.cpp

调用方式:

```
ros2 run pgo pcd_to_nav2_map \
  --pcd /path/to/map.pcd \
  --out-dir /path/to/output \
  [可选参数]
```

输出文件:

- map.pgm: PGM 灰度栅格图像
- map.yaml: Nav2 地图服务器所需的 YAML 元数据

2.5.2 转换算法 (三阶段流水线)

阶段一: Z 轴切片投影

将 3D 点云投影到 XY 平面, 对每个 2D 栅格单元统计三类命中:

observed_hits: [z_min, z_max] 范围内的点 → 表示该区域被观测过

obstacle_hits: [obstacle_z_min, obstacle_z_max] 范围内的点 → 机器人高度范围内的障碍

ceiling_hits: (obstacle_z_max, z_max] 范围内的点 → 天花板/高处结构

栅格分类逻辑:

```
对每个栅格单元：
  若 observed_hits > 0      → 标记为 FREE
  若 ceiling_hits > 0 且 obstacle_hits == 0 → 标记为 FREE
  若 obstacle_hits >= obstacle_min_hits → 标记为 OCCUPIED
```

阶段二：滤波与清理

- 1. **SOR 统计离群点移除**：对障碍高度范围内的点做统计滤波
- 2. **射线投射 (Raycasting)**：从传感器位姿到观测点画射线，沿途标记为 FREE（Bresenham 直线算法）
- 3. **小聚类移除**：用 BFS 找连通占据区域，面积过小的删除

阶段三：膨胀

对所有占据栅格做圆形膨胀（半径 = `inflation_radius`），为机器人足迹提供安全裕量。

2.5.3 完整参数表

参数	默认值	说明
<code>--resolution</code>	0.05 m	栅格分辨率
<code>--z-min</code>	-0.30 m	观测点 Z 轴下界
<code>--z-max</code>	2.20 m	观测点 Z 轴上界
<code>--obstacle-z-min</code>	0.08 m	障碍物 Z 轴下界
<code>--obstacle-z-max</code>	1.80 m	障碍物 Z 轴上界
<code>--inflation-radius</code>	0.15 m	占据栅格膨胀半径
<code>--padding</code>	1.0 m	地图边界额外边距
<code>--obstacle-min-hits</code>	1	标记为占据的最少点数
<code>--sor-mean-k</code>	0（关闭）	SOR 近邻分析数量
<code>--sor-stddev</code>	1.0	SOR 标准差阈值
<code>--min-obstacle-cluster</code>	0（关闭）	最小占据聚类大小
<code>--poses</code>	无	传感器位姿文件（用于射线投射）

2.5.4 PGM 输出编码

灰度值	含义
0	占据（黑色）
254	自由（白色）
205	未知（灰色）

2.5.5 室外建议参数

```
ros2 run pgo pcd_to_nav2_map \
  --pcd map.pcd --out-dir output \
  --obstacle-z-min 0.15 \
  --resolution 0.10 \
  --sor-mean-k 20 --sor-stddev 1.0 \
  --min-obstacle-cluster 5 \
  --poses poses.txt
```

第三章 定位

3.1 定位快速开始

完成建图后，定位由 `icp_registration` 加载 3D `map.pcd`，实时点云与地图配准后发布 `map -> odom`；`safe_odom_mux` 发布连续的 `odom -> base_link`；两者合成得到机器人在地图中的 `map -> base_link`。

3.1.1 启动定位与导航链路

```
export MAP_NAME=your_map

ros2 launch swerve_navigation full_system.launch.py \
  pcd_path:="$HOME/MID_360_nav/openflex_maps/3d/$MAP_NAME/map.pcd" \
```

```
map_yaml:="$HOME/MID_360_nav/openflex_maps/2d/$MAP_NAME/map.yaml" \  
initial_pose:"[0.0,0.0,0.0,0.0,0.0,0.0]"
```

参数	作用
pcd_path	ICP 定位使用的 3D 点云地图
map_yaml	Nav2 路径规划使用的 2D 栅格地图
initial_pose	ICP 初始位姿猜测 [x,y,z,roll,pitch,yaw]

3.1.2 定位核心关系

```
3D map.pcd + 当前点云 /fastlio2/body_cloud  
-> icp_registration  
-> map -> odom  
  
LIO/LIVO 里程计 /fastlio2/lio_odom + 轮式里程计 /odom  
-> safe_odom_mux  
-> odom -> base_link  
  
map -> base_link = map -> odom × odom -> base_link
```

数据/模块	作用
map.pcd	已保存的 3D 点云地图，供 ICP 全局配准
/fastlio2/body_cloud	当前实时点云
icp_registration_node	将当前点云对齐到 3D 地图，发布 map -> odom
/fastlio2/lio_odom 和 /odom	提供短期连续运动估计
safe_odom_mux.py	发布稳定的 /odom_safe 和 odom -> base_link

3.1.3 简便判断定位是否准确

在 RViz 里看实时点云是否和地图重合：

- 机器人现实中在门口、墙角、柱子旁时，RViz 中机器人模型也应在地图对应位置。
- /fastlio2/body_cloud 的墙面、门框等结构应与已加载地图基本重合。
- 大概可用时，位置误差通常应小于 20-30 cm，朝向误差小于约 10 度。

3.1.4 常用检查命令

```
ros2 run tf2_ros tf2_echo map base_link  
ros2 topic hz /fastlio2/body_cloud  
ros2 topic hz /fastlio2/lio_odom  
ros2 topic hz /odom_safe
```

3.1.5 初始位姿不准时怎么办

用 RViz 的 2D Pose Estimate 在地图上点击机器人当前位置和朝向，或在启动时给更接近真实位置的 initial_pose。也可以命令行手动发布：

```
ros2 topic pub --once /initialpose \  
  geometry_msgs/PoseWithCovarianceStamped \  
  '{header: {frame_id: map}, pose: {pose: {position: {x: 1.0, y: 2.0, z: 0}, orientation: {w: 1.0}}}}'
```

3.1.6 定位问题简易排查

现象	优先处理
初始定位失败	用 RViz 2D Pose Estimate 给更接近真实位置的初值
定位漂移或跳变	检查实时点云与 3D 地图是否重合
长走廊定位不稳	降低速度，让雷达看到几何特征
重定位识别失败	检查 sc_data/ 目录是否存在

3.2 导航模式下的 LIO 与 LIVO 里程计

3.2.1 角色

导航模式下，`full_system.launch.py` 使用 `fast_livo` 包的 `swerve_livo.launch.py`，节点命名空间保持为 `fastlio2` 以兼容既有话题。该链路输出短期 LIO/LIVO 里程计 `/fastlio2/lio_odom` 和点云 `/fastlio2/body_cloud`；随后 `safe_odom_mux.py` 在 LIO/LIVO 里程计和轮式里程计 `/odom` 之间做安全切换，发布 `/odom_safe` 和 `odom → base_link TF`。

导航配置文件：

- LIO/LIVO 核心：`src/openflex_chassis/mapping_localization_layer/fast_livo/config/livo_navigation.yaml`
- 安全里程计复用器：`src/openflex_chassis/navigation_layer/swerve_navigation/scripts/safe_odom_mux.py`

3.2.2 输入输出

方向	话题/TF	消息类型	频率
输入	<code>/livox/imu</code>	<code>sensor_msgs/Imu</code>	200 Hz
输入	<code>/livox/lidar</code>	<code>livox_ros_driver2/CustomMsg</code>	10 Hz
LIO/LIVO 输出	<code>/fastlio2/lio_odom</code>	<code>nav_msgs/Odometry</code>	10 Hz
LIO/LIVO 输出	<code>/fastlio2/body_cloud</code>	<code>sensor_msgs/PointCloud2</code>	10 Hz
控制器输出	<code>/odom</code>	<code>nav_msgs/Odometry</code>	50 Hz
<code>safe_odom_mux</code> 输出	<code>/odom_safe</code>	<code>nav_msgs/Odometry</code>	50 Hz
<code>safe_odom_mux</code> 输出	<code>odom → base_link TF</code>	TF	50 Hz

3.2.3 漂移特性

距离	典型漂移	说明
< 50 m	< 几厘米	几乎可忽略
100~500 m	5~20 cm	需要 ICP 校正
退化场景	可能更大	长走廊、空旷区域特征不足

3.3 ICP 点云配准定位

3.3.1 定位方案概述

当前工作区默认使用 `icp_registration_node` 进行 PCD 地图配准定位。

特性	<code>icp_registration_node</code>
包名	<code>icp_registration</code>
代码位置	<code>src/openflex_chassis/mapping_localization_layer/icp_registration/</code>
配置文件	<code>src/openflex_chassis/mapping_localization_layer/icp_registration/config/icp.yaml</code>
输入	PointCloud2 点云
TF 输出	<code>map → odom</code>
重定位入口	<code>/initialpose</code> 话题
初始搜索	ScanContext + NDT + ICP，失败后降级到网格 ICP
持续重对齐	支持，可配置间隔

3.3.2 两阶段配准算法

阶段一：粗配准 (Rough Alignment)

1. 对输入点云做体素下采样 (`leaf_size = 0.4m`)

2. 生成多假设初始猜测：

├ XY 方向：在 `±xy_offset` (`5.0m`) 内生成位置候选

└ Yaw 方向：在 `±yaw_offset` (`180°`) 内按 `yaw_resolution` (`30°`) 生成角度候选

→ 共约 117 个候选位姿

3. 对每个候选执行 ICP (最多 `rough_iter=30` 次迭代)

4. 选择得分最佳且 `< 2×thresh` 的结果

阶段二：精配准 (Fine Alignment)

1. 对输入点云做体素下采样 (`leaf_size = 0.1m`)

2. 以粗配准结果为初始猜测

3. 执行 ICP (最多 refine_iter=20 次迭代)

4. 得分 < thresh (0.5) 则接受

map→odom 计算公式:

$$\text{map}\rightarrow\text{odom} = \text{map}\rightarrow\text{laser} \times (\text{odom}\rightarrow\text{laser})^{-1}$$

3.3.3 ICP 参数表

参数	值	说明
map_frame_id	map	地图坐标帧
odom_frame_id	odom	里程计坐标帧
base_frame_id	base_link	机器人基座帧
laser_frame_id	base_link	点云帧
rough_leaf_size	0.4 m	粗配准体素大小
refine_leaf_size	0.1 m	精配准体素大小
thresh	0.5	ICP 适配度阈值
rough_iter	30	粗配准最大迭代次数
refine_iter	20	精配准最大迭代次数
xy_offset	5.0 m	初始搜索 XY 范围
yaw_offset	180.0°	初始搜索 Yaw 范围
yaw_resolution	30.0°	Yaw 搜索步长
continuous_realign_interval_sec	5.0 s	持续重对齐间隔
tf_future_tolerance_sec	0.2 s	TF 时间戳前移量
initial_pose	[0,0,0,0,0]	初始位姿

NDT 参数 (用于 ScanContext 粗配准) :

参数	值	说明
ndt_resolution	1.0 m	NDT 栅格分辨率
ndt_step_size	0.1 m	NDT 步长
ndt_max_iterations	30	NDT 最大迭代次数
ndt_epsilon	0.01	NDT 收敛阈值

ScanContext 参数:

参数	值	说明
submap_half_range	5	子地图半径
sc_dist_thresh	0.4	ScanContext 相似度阈值

3.4 ScanContext 回环检测与重定位

3.4.1 算法原理

ScanContext 是一种基于极坐标描述子的全局位置识别算法。

代码文件位置:

- src/openflex_chassis/mapping_localization_layer/icp_registration/third_party/scancontext/Scancontext.h
- src/openflex_chassis/mapping_localization_layer/icp_registration/third_party/scancontext/Scancontext.cpp

3.4.2 描述子构建

将 3D 点云转换为极坐标网格:

1. 将点云投影到极坐标 (r, θ)

2. 划分为 PC_NUM_RING=20 个环 × PC_NUM_SECTOR=60 个扇区

3. 每个网格单元取最大 Z 值（高度）

4. 得到 20×60 的矩阵作为场景描述子

参数	值	说明
PC_NUM_RING	20	径向环数
PC_NUM_SECTOR	60	角度扇区数
PC_MAX_RADIUS	80.0 m	最大检测半径

3.4.3 匹配算法

1. 降维：沿行求均值得到 RingKey（1×20 向量）

2. 通过 KD 树（nanoflann）搜索 RingKey 最近邻

3. 对每个候选：

a) 通过列均值（SectorKey）快速预对齐旋转

b) 计算余弦距离（考虑所有旋转偏移）

c) 距离 < SC_DIST_THRES（0.4）则接受

4. 返回最佳匹配的关键帧 ID 和旋转偏移

3.4.4 ScanContext 数据库

建图阶段 PGO 保存地图时同时保存 ScanContext 数据库：

```
/path/to/3d_map/  
├─ map.pcd  
├─ poses.txt  
└─ sc_data/  
    ├─ scancontext_0.bin  
    ├─ scancontext_1.bin  
    └─ ...
```

3.5 ICP 三级定位策略

icp_registration_node 采用三级降级的定位策略：

3.5.1 第一级：ScanContext + NDT + ICP

触发条件：收到 /initialpose 话题消息

1. ScanContext 在数据库中匹配当前扫描 → 得到候选关键帧 ID

2. 提取该关键帧周围 ±submap_half_range 个帧构建子地图

3. NDT 粗配准：以 ScanContext 预对齐结果为初始猜测

4. ICP 精配准：以 NDT 结果为初始猜测

5. 如果 ICP 得分 < thresh → 成功，发布 map-odom

典型耗时：~3-5 秒

3.5.2 第二级：ScanContext + ICP（跳过 NDT）

触发条件：第一级中 NDT 配准失败

1. ScanContext 匹配成功但 NDT 粗配准失败

2. 直接使用 ScanContext 关键帧位姿作为 ICP 初始猜测

3. ICP 粗配准 → ICP 精配准

4. 如果 ICP 得分 < thresh → 成功

3.5.3 第三级：网格搜索 ICP

触发条件：第一级和第二级都失败，或 ScanContext 数据库为空

1. 以当前位姿估计为中心

2. 在 ±xy_offset × ±yaw_offset 范围内生成约 117 个候选位姿

3. 对每个候选执行 ICP 粗配准 → 精配准

4. 选择全局最佳匹配

典型耗时：~10-15 秒

3.5.4 降级流程图



3.6 ICP 持续重对齐机制

即使初始定位成功，长时间运行后 odom 漂移会逐渐增大。持续重对齐机制定期修正 map→odom 变换。

每 continuous_realign_interval_sec (5.0s) 检查一次：

如果 is_ready_ == true 且 alignment_in_progress_ == false：

1. 以当前 map→odom 位姿为初始猜测

2. 跳过 ScanContext (skip_sc=true)，直接执行 ICP 精配准

3. 如果收敛 → 更新 map→odom

4. 如果不收敛 → 保持上一次有效位姿

关键设计点：

- 持续重对齐时位姿偏移量小，不需要全局重定位，直接 ICP 最快
- 非阻塞执行：在独立线程中运行
- 失败不丢位：保持上一次成功的 map→odom

3.7 TF 树与坐标系关系

3.7.1 导航模式完整 TF 树



3.7.2 各 TF 变换的提供者

变换	提供者	频率	算法
map → odom	icp_registration_node	~100 Hz	ICP 点云配准
odom → base_link	safe_odom_mux	50 Hz	LIO/LIVO 与轮式 odom 安全切换
base_link → mid360_link	robot_state_publisher	静态	URDF 固定关节
base_link → *_steering_link	robot_state_publisher	随关节更新	URDF + joint_states

3.7.3 两层定位架构

第一层：短期里程计（高频、平滑、会漂移）	
safe_odom_mux: /fastlio2/lio_odom + /odom → TF	
频率：50Hz LIO异常时临时使用轮式增量	
第二层：全局定位（低频、校正漂移）	
ICP: map → odom	
频率：~100Hz 延迟：~50ms 漂移：持续修正	

Nav2 使用 `map → base_link` 的复合变换，兼具高频平滑和全局准确。

3.8 定位精度与漂移分析

3.8.1 正常工作状态

指标	典型值
位置精度	5~10 cm
朝向精度	1~3°
ICP 收敛时间	< 100 ms
定位丢失概率	极低

3.8.2 可能导致定位失败的场景

场景	原因	缓解方法
环境变化大	家具移动、物品增减	重新建图
特征退化	长走廊、空旷区域	确保有足够几何特征
剧烈运动	快速旋转	降低最大角速度
LIDAR 遮挡	点云质量差	保持传感器清洁
初始位姿偏差大	开机位置与地图不符	使用 /initialpose 手动指定

第四章 导航

4.1 导航快速开始

`full_system.launch.py` 是 OpenFlex 的完整导航入口，会同时启动底盘控制、传感器、定位、Nav2、碰撞监控和 RViz。

4.1.1 启动命令

```
export MAP_NAME=your_map

ros2 launch swerve_navigation full_system.launch.py \
  pcd_path:="$HOME/MID_360_nav/openflex_maps/3d/$MAP_NAME/map.pcd" \
  map_yaml:="$HOME/MID_360_nav/openflex_maps/2d/$MAP_NAME/map.yaml" \
  initial_pose="[0.0,0.0,0.0,0.0,0.0,0.0]" \
  use_rviz:=true
```

Launch 文件位置： `src/openflex_chassis/navigation_layer/swerve_navigation/launch/full_system.launch.py`

参数	默认值	说明
pcd_path	必填	ICP 定位用的 3D PCD 地图文件
map_yaml	必填	Nav2 使用的 2D 栅格地图 YAML 文件
initial_pose	[0,0,0,0,0,0]	ICP 初始位姿
use_rviz	true	是否启动 RViz
steering_can_interface	can5	转向电机 CAN 接口
driving_can_interface	can4	驱动电机 CAN 接口
nav_controller_max_wheel_speed	1.2	导航模式轮速上限
nav_controller_wheel_accel_limit	1.0	导航模式轮速变化率上限

4.1.2 RViz 使用方法

1. 确认地图已显示，机器人模型在地图附近。
2. 使用 `2D Pose Estimate` 点击机器人实际位置和朝向。
3. 观察实时点云是否与地图重合。
4. 定位正确后，使用 `Nav2 Goal` 点击目标位置。
5. 观察全局路径、局部路径和 `/cmd_vel_safe`。

4.1.3 运行过程检查

```
ros2 run tf2_ros tf2_echo map base_link
ros2 topic hz /odom_safe
ros2 topic hz /scan
ros2 topic echo --once /cmd_vel_safe
```

现象	判断
<code>map -> base_link</code> 持续发布	全局定位链路在工作
<code>/scan</code> 有稳定频率	实时避障输入正常
<code>Nav2 Goal</code> 后有路径并输出 <code>/cmd_vel_safe</code>	Nav2 控制链路在工作

4.1.4 导航效果调试

现象	优先处理
机器人整体太快	降低 <code>desired_linear_vel</code> 、Velocity Smoother 的 <code>max_velocity</code>
运动不够平稳	降低 <code>max_accel</code> 、 <code>max_decel</code>
路径跟随不紧密	提高 NMPC 权重 <code>Q_px</code> 、 <code>Q_py</code>
太靠近障碍物	增大 local costmap <code>inflation_radius</code>
窄通道过不去	减小 <code>inflation_radius</code> ，检查 <code>footprint</code>

4.2 Nav2 整体架构

配置文件位置：`src/openflex_chassis/navigation_layer/swerve_navigation/config/nav2_params.yaml`

4.2.1 组件列表

服务器	功能	插件
BT Navigator	行为树导航决策	Nav2 行为树节点集
Controller Server	局部路径跟踪	<code>nmpc_controller::NmpcController</code>
Planner Server	全局路径规划	<code>nav2_smac_planner/SmacPlanner2D</code>
Smoother Server	路径平滑	SimpleSmoother
Behavior Server	恢复行为	Spin, Backup, Wait
Velocity Smoother	速度平滑	VelocitySmoother
Waypoint Follower	航点跟随	WaypointFollower

4.2.2 导航核心参数

参数	值	说明
<code>global_frame</code>	<code>map</code>	全局参考帧
<code>robot_base_frame</code>	<code>base_link</code>	机器人帧
<code>odom_topic</code>	<code>/odom_safe</code>	安全里程计
<code>bt_loop_duration</code>	20 ms	行为树循环周期
<code>default_server_timeout</code>	20 s	服务器动作超时
<code>controller_frequency</code>	20.0 Hz	控制器执行频率

4.3 NMPC 局部控制器

4.3.1 算法原理

当前项目使用自定义 Nav2 Controller 插件 `nmmpc_controller::NmmpcController`。该控制器把舵轮底盘抽象成可全向平移的二阶非线性模型，每个控制周期求解有限时域非线性最优控制问题。

```
state  x = [px, py, theta, vx, vy, omega]
control u = [ax, ay, alpha]
params p = [xref(6), costmap_cost]

dpx   = vx*cos(theta) - vy*sin(theta)
dpy   = vx*sin(theta) + vy*cos(theta)
dtheta = omega
dvx   = ax
dvy   = ay
domega = alpha
```

求解器由 acados/CasADi 生成，默认预测时域 `N=20`，`Tf=2.0s`，使用 SQP_RTI、HPIPM QP、ERK 积分和 Gauss-Newton Hessian 近似。

代码文件位置：

- 控制器：`src/openflex_chassis/navigation_layer/nmmpc_controller/src/nmmpc_controller.cpp`
- 求解器生成脚本：`src/openflex_chassis/navigation_layer/nmmpc_controller/script/generate_solver.py`

4.3.2 优化目标与权重

权重	值	说明
<code>Q_px, Q_py</code>	65.0, 65.0	位置跟踪
<code>Q_theta</code>	8.0	航向跟踪
<code>Q_vx, Q_vy, Q_omega</code>	1.5, 0.8, 2.0	速度参考跟踪
<code>R_ax, R_ay, R_alpha</code>	0.08, 0.10, 0.12	控制加速度惩罚
<code>Q_e_px, Q_e_py, Q_e_theta</code>	85.0, 85.0, 12.0	终端位置/航向
<code>costmap_weight</code>	60.0	障碍代价权重

4.3.3 速度限制

参数	值	说明
<code>vx_max</code>	0.50 m/s	最大前进速度
<code>vx_min</code>	-0.20 m/s	最大后退速度
<code>vy_max</code>	0.25 m/s	最大横向速度
<code>omega_max</code>	1.0 rad/s	最大角速度
<code>ax_max</code>	0.8 m/s²	前向加速度约束
<code>ay_max</code>	0.6 m/s²	横向加速度约束
<code>alpha_max</code>	1.8 rad/s²	角加速度约束

4.3.4 舵轮 native 路径跟踪策略

参数	值	说明
<code>allow_lateral_tracking</code>	true	允许全向平移
<code>swerve_native_mode</code>	true	车头朝终点方向，底盘横移贴路径
<code>start_heading_capture_dist</code>	0.80 m	起步捕获距离
<code>final_rotate_xy_tolerance</code>	0.18 m	终点原地旋转 XY 容差
<code>final_rotate_yaw_goal_tolerance</code>	0.08 rad	终点航向容差
<code>lio_odom_topic</code>	<code>/fastlio2/lio_odom</code>	LIO twist 初始化 NMPC 状态

4.3.5 局部 A* 避障

控制器内部带短距离 A* 避障补丁，在局部路径前方出现障碍时快速绕行：

1. 沿局部路径检查前方 `local_astar_obstacle_lookahead` 范围内的代价
2. 若连续路径点代价 `>= local_astar_cost_threshold`，判定阻塞
3. 在 `odom costmap` 栅格上做 8 邻域 A*
4. 对绕行路径做平滑，再拼回局部路径

参数	值	说明
local_astar_enabled	true	是否启用
local_astar_cost_threshold	0.40	触发绕行的代价
local_astar_reconnect_dist	1.5 m	障碍后重连距离
local_astar_obstacle_lookahead	2.5 m	前方检查距离

4.3.6 目标容差

参数	值	说明
xy_goal_tolerance	0.18 m	Nav2 GoalChecker 位置容差
yaw_goal_tolerance	0.15 rad	Nav2 GoalChecker 航向容差
required_movement_radius	0.06 m	需移动的最小距离
movement_time_allowance	15.0 s	允许的最长不移动时间

4.4 全局路径规划器

4.4.1 SmacPlanner2D

参数	值	说明
plugin	nav2_smac_planner/SmacPlanner2D	2D A* 规划器
tolerance	0.10 m	目标容差
allow_unknown	true	允许穿过未知区域
max_iterations	1000000	最大搜索迭代
max_planning_time	2.0 s	最大规划耗时
cost_travel_multiplier	10.0	代价地图权重

4.4.2 路径平滑器

参数	值	说明
tolerance	1.0e-10	收敛容差
max_its	1000	最大迭代次数
do_refinement	true	是否精化处理

4.5 代价地图配置

4.5.1 局部代价地图（Local Costmap）

参数	值	说明
update_frequency	10.0 Hz	更新频率
publish_frequency	5.0 Hz	发布频率
width	5 m	地图宽度
height	5 m	地图高度
resolution	0.05 m	栅格分辨率
rolling_window	true	跟随机器人滚动
global_frame	odom	参考帧

机器人足迹：

```
footprint: "[[-0.37, -0.31], [-0.37, 0.31], [0.37, 0.31], [0.37, -0.31]]"
```

即 0.74m × 0.62m 的矩形（含安全裕量）。

障碍物层参数：

参数	值	说明
----	---	----

参数	值	说明
observation_sources	scan	数据源为 /scan
max_obstacle_height	2.0 m	最大障碍物高度
obstacle_max_range	4.5 m	标记障碍物最大距离
raytrace_max_range	5.0 m	射线清除最大距离

膨胀层参数：

参数	值	说明
inflation_radius	0.30 m	障碍物膨胀半径
cost_scaling_factor	3.0	代价衰减速度

4.5.2 全局代价地图（Global Costmap）

参数	值	说明
update_frequency	5.0 Hz	更新频率
publish_frequency	2.0 Hz	发布频率
resolution	0.05 m	栅格分辨率
global_frame	map	参考帧
track_unknown_space	true	跟踪未知区域

全局代价地图额外包含 Static Layer，加载预生成 2D 地图。

4.5.3 代价地图层叠结构



4.6 速度平滑器

对 NMPC 输出的速度命令做加速度/减速度限制，防止电机过载和底盘打滑。

参数	值	说明
smoothing_frequency	20.0 Hz	平滑频率
scale_velocities	true	超限时等比缩放速度
feedback	OPEN_LOOP	开环
max_velocity	[0.50, 0.22, 1.0]	[vx, vy, wz] 最大值
min_velocity	[-0.20, -0.22, -1.0]	[vx, vy, wz] 最小值
max_accel	[0.8, 0.6, 1.5]	最大加速度
max_decel	[-1.0, -0.6, -1.8]	最大减速度
velocity_timeout	1.0 s	速度命令超时自动停车

4.7 行为服务器与恢复行为

当导航遇到困难时，行为树触发恢复行为：

行为	说明
spin	原地旋转，清除周围障碍信息
backup	向后倒车
drive_on_heading	沿当前方向直行
wait	等待动态障碍物离开

行为	说明
<code>assisted_teleop</code>	辅助遥控

行为服务器参数：

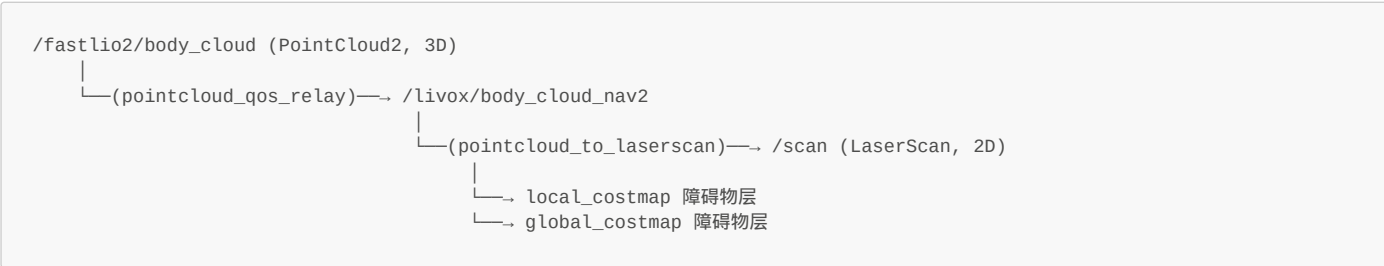
参数	值	说明
<code>cycle_frequency</code>	10.0 Hz	行为执行频率
<code>simulate_ahead_time</code>	2.0 s	碰撞模拟前瞻时间
<code>max_rotational_vel</code>	1.0 rad/s	Spin 最大角速度
<code>min_rotational_vel</code>	0.4 rad/s	Spin 最小角速度

4.8 3D 点云转 2D 激光扫描（实时）

4.8.1 概述

`pointcloud_to_laserscan` 节点在导航运行时持续将 3D 点云投影为 2D 激光扫描，供局部代价地图使用。

数据流：



4.8.2 参数

参数	导航模式值	说明
输入点云	<code>/livox/body_cloud_nav2</code>	经过 relay 自车滤波
<code>target_frame</code>	<code>base_link</code>	投影参考帧
<code>min_height</code>	-0.15 m	截取点云下界
<code>max_height</code>	0.40 m	截取点云上界
<code>angle_min</code>	-n	扫描起始角
<code>angle_max</code>	n	扫描结束角（360°）
<code>angle_increment</code>	0.0043 rad	角分辨率
<code>range_min</code>	0.45 m	最小有效距离
<code>range_max</code>	10.0 m	最大有效距离

4.8.3 算法说明

- 接收 3D 点云
 - 变换到 `target_frame (base_link)`
 - 按 `min_height/max_height` 过滤 Z 轴
 - 按 `angle_increment` 将点分入角度 bin
 - 每个 bin 取最近的点
 - 输出 2D LaserScan

第五章 全身VLA大模型VR数据采集、训练与推理

5.1 全身 VLA 快速开始

本节用于快速跑通 OpenFlex 的全身 VLA 数据采集与推理链路。VLA 不建议直接学习长距离导航；推荐让 Nav2/NMPC 负责长程移动和避障，VLA 主要学习到位后的底盘微调、升降、头部观察和双臂操作。

注意：VLA 数据采集、训练与推理属于可选扩展模块，当前 OpenFlex 基础源码包不包含 `openflex_vla` 实现。下面涉及 `<openflex_vla_package>` 的命令需要在安装对应 VLA 扩展包后，替换为该扩展包实际提供的 ROS 包名。

5.1.1 启动全身硬件、SLAM 与相机

```
cd ~/openflex_all/openflex_ws
source install/setup.bash

ros2 launch <openflex_vla_package> wholebody_vla_record.launch.py
```

该 launch 启动整机硬件、Livox MID-360、FAST-LIO2 odom-only 链路和四路相机。

5.1.2 启动 VR 遥操作

```
ros2 launch openarmx_integrated_bringup integrated_vr_teleop.launch.py
```

采集底盘数据时建议降低 VR 底盘速度：

```
ros2 launch openarmx_integrated_bringup integrated_vr_teleop.launch.py \
  vr_max_linear_speed:=0.2 \
  vr_boost_linear_speed:=0.35 \
  vr_max_angular_speed:=0.5 \
  vr_linear_expo:=2.0 \
  vr_angular_expo:=2.5
```

5.1.3 录制数据集

```
python3 scripts/lerobot_record_openflex.py \
  --robot.type=openarmx_follower_ros2 \
  --robot.skip_send_action=true \
  --robot.ros2.enable_base=true \
  --robot.ros2.enable_head=true \
  --robot.ros2.enable_lift=true \
  --robot.ros2.odom_topic=/fastlio2/lio_odom \
  --robot.ros2.base_action_mode=pose \
  --robot.ros2.base_path_coordinate_mode=episode_local \
  --teleop.type=openarmx_leader_ros2 \
  --teleop.ros2.enable_base=true \
  --teleop.ros2.enable_head=true \
  --teleop.ros2.enable_lift=true \
  --teleop.ros2.odom_topic=/fastlio2/lio_odom \
  --teleop.ros2.base_action_mode=pose \
  --teleop.ros2.base_path_coordinate_mode=episode_local \
  --dataset.repo_id=YOUR_NAME/openflex_walk_and_grasp \
  --dataset.num_episodes=50 \
  --dataset.fps=15 \
  --dataset.episode_time_s=60 \
  --dataset.reset_time_s=10 \
  --dataset.single_task="Walk to the table and pick up the cup"
```

5.1.4 推理部署注意事项

推理前必须关闭 VR 遥操作，保留整机 bringup、FAST-LIO2 odom 和相机。默认 `pose/trajectory` 模式下，策略输出底盘未来位姿或轨迹，Follower 发布 `/vla/base_path`，底盘 `path` 跟踪链路生成实际 `/cmd_vel`。

5.2 LeRobot 接入与全身采集目标

OpenFlex 的 VLA 数据链路采用 LeRobot 的 `Robot/Follower` 与 `Teleoperator/Leader` 抽象。采集时机器人由 VR 遥操作真实闭环控制，LeRobot 只记录观测和示教动作；推理时再把训练出的 `action` 写回 ROS2 控制话题。

VR 遥操作节点		
—	/left_forward_position_controller/commands	-> 左臂 action 标签
—	/right_forward_position_controller/commands	-> 右臂 action 标签
—	/head_forward_position_controller/commands	-> 头部 action 标签
—	/velocity_controller/commands	-> 升降台 action 标签
—	/cmd_vel	-> 底盘示教速度

实现位置：

角色	路径	作用
Follower / Robot	external openflex_vla package/lerobot_robot_openflex_follower_ros2/	读取观测，推理时发送动作

角色	路径	作用
Leader / Teleoperator	external openflex_vla package/lerobot_teleoperator_openflex_leader_ros2/	读取 VR 控制命令并生成 action 标签
全身采集 launch	external openflex_vla package: launch/wholebody_vla_record.launch.py	整机硬件、FAST-LIO2 odom、相机
本地录制脚本	scripts/lerobot_record_openflex.py	校验底盘 schema，episode 开始重置相对 odom
推理脚本	scripts/lerobot-infer-openflex.py	加载策略并调用 Follower send_action()

5.3 OpenFlex 全身数据 Schema

全身模式打开 enable_base=True, enable_head=True, enable_lift=True。默认底盘 action 模式为 base_action_mode=pose，坐标语义为 base_path_coordinate_mode=episode_local。

5.3.1 Observation（26 个标量 + 4 路 RGB）

维度	Key	来源	说明
8	openarmx_left_joint{1..7}.pos + openarmx_left_finger_joint1.pos	/joint_states	左臂 7 关节 + 夹爪
8	openarmx_right_joint{1..7}.pos + openarmx_right_finger_joint1.pos	/joint_states	右臂 7 关节 + 夹爪
2	openarmx_head_yaw_joint.pos, openarmx_head_pitch_joint.pos	/joint_states	头部 yaw / pitch
2	lift_joint.pos, lift_joint.vel	/joint_states	升降台位置与速度
3	x.vel, y.vel, theta.vel	/fastlio2/lio_odom.twist	底盘机体系速度
3	odom_x, odom_y, odom_theta	/fastlio2/lio_odom.pose	episode-local 位姿
4 张图	right_wrist, left_wrist, head, base	ROS2 CompressedImage	RGB 图像

相机默认配置：

Key	话题	分辨率	帧率
right_wrist	/cam_right/color/image/compressed	320x240	15 Hz
left_wrist	/cam_left/color/image/compressed	320x240	15 Hz
head	/cam_head/color/image/compressed	640x480	15 Hz
base	/cam_base/color/image/compressed	320x240	15 Hz

5.3.2 Action（底盘三种模式）

双臂、头部、升降台 action 在三种底盘模式下保持一致：

维度	Key	来源话题	控制语义
8	左臂各 *.pos	/left_forward_position_controller/commands	左臂关节目标位置
8	右臂各 *.pos	/right_forward_position_controller/commands	右臂关节目标位置
2	头部各 *.pos	/head_forward_position_controller/commands	头部 yaw / pitch
1	lift_joint.vel	/velocity_controller/commands	升降台速度命令

底盘 action 由 base_action_mode 决定：

模式	底盘 key	总 action 维度	推理时执行方式
pose（默认）	base_target_x/y/theta	22	发布 /vla/base_path
trajectory	base_path_00~07_x/y/theta	43	发布 8 点 Path
twist	base_vx/vy/wz	22	直接发布 /cmd_vel

5.3.3 Schema 一致性约束

字段	要求
odom_topic	两侧一致，默认 /fastlio2/lio_odom
base_action_mode	两侧一致，默认 pose
base_trajectory_points	两侧一致，默认 8
base_path_coordinate_mode	两侧一致，默认 episode_local

5.4 底盘观测与动作参考对比

主流移动操作/VLA 数据集很少直接学习轮速、轮矩或电机电流，通常把底盘抽象为 base/chassis twist、相对位姿增量或短时轨迹。

数据集/项目	底盘 observation	底盘 action
Galaxea Open-World / G0	chassis state、velocities、IMU	chassis velocities (6D twist)
AgiBot World Alpha	robot position、orientation	robot velocity [vx, wz]
RoboChallenge ICRA WBC	robot position、orientation	6 维 mobile base command
Mobile ALOHA	图像与双臂关节	14D ALOHA + base linear/angular
RT-1	视觉历史和语言	3D base movement x, y, yaw

对 OpenFlex 的设计决策：

1. 不采集轮速作为 VLA 主输入。底盘是四转四驱，底层控制器细节不应暴露给策略。
2. 保留速度观测。x.vel/y.vel/theta.vel 反映底盘运动状态。
3. 加入 episode-local pose。每条 episode 重置局部原点，避免全局漂移。
4. 默认 action 选 pose。策略学习"下一步到哪里"，推理时由底盘闭环执行。
5. 长程导航仍交给 Nav2。VLA 集中在到位后的底盘微调 and 双臂操作。

5.5 Follower (Robot) 实现

注册名：openflex_follower_ros2，兼容别名 openarmx_follower_ros2。

```
robot_config = OpenFlexFollowerRos2Config(
    skip_send_action=True,
    relative_odom=True,
    ros2=OpenFlexRos2InterfaceConfig(
        enable_base=True,
        enable_head=True,
        enable_lift=True,
        odom_topic="/fastlio2/lio_odom",
        base_action_mode="pose",
        base_path_coordinate_mode="episode_local",
        base_path_topic="/vla/base_path",
        base_path_frame_id="odom",
    ),
)
```

参数	默认值	说明
skip_send_action	True	录制时只读观测，不向机器人发命令
relative_odom	True	每个 episode 使用局部 odom 原点
odom_wait_timeout_s	5.0	等待 odom 话题超时
base_action_mode	pose	推理时把底盘 action 转成 /vla/base_path

5.6 Teleoperator (Leader) 实现

注册名：openflex_leader_ros2，兼容别名 openarmx_leader_ros2。

订阅话题	消息类型	写入 action
/left_forward_position_controller/commands	Float64MultiArray	左臂 8 维
/right_forward_position_controller/commands	Float64MultiArray	右臂 8 维
/head_forward_position_controller/commands	Float64MultiArray	头部 2 维
/velocity_controller/commands	Float64MultiArray	升降台
/cmd_vel	Twist	底盘 action
/fastlio2/lio_odom	Odometry	积分 episode-local 目标

5.7 全身数据录制流程

5.7.1 录制前检查


```
ros2 topic hz /joint_states
ros2 topic hz /fastlio2/lio_odom
ros2 topic hz /cmd_vel
ros2 topic hz /cam_left/color/image/compressed
ros2 topic hz /cam_right/color/image/compressed
ros2 topic hz /cam_head/color/image/compressed
ros2 topic hz /cam_base/color/image/compressed
```

5.7.2 数据规模建议

阶段	数据量	内容
Pipeline 验证	5~10 episodes	仅双臂，确认 schema
固定站位全身操作	50~100 episodes	底盘小幅对位、升降、双臂抓取
短程移动操作	100~300 episodes	1~3 m 移动后对位和操作
多场景泛化	300+ episodes	不同桌面、高度、物体、光照

5.8 数据质量检查与标定要求

检查项	要求	原因
相机时间与帧率	4 路图像稳定接近 15 Hz	避免 action 与视觉错位
<code>/fastlio2/lio_odom</code>	pose/twist 连续，无明显跳变	底盘 pose action 和 observation 都依赖该源
episode reset	每条 episode 开头 <code>odom_x/y/theta</code> 接近 0	保证局部坐标语义一致
VR 命令话题	控制时有频率，松手后归零	防止记录 stale action
任务语言	同一任务使用稳定、具体的自然语言	便于 VLA 训练语言条件

常见故障：

现象	原因	处理
<code>Odom not available yet</code>	FAST-LIO2 未启动	检查 <code>/fastlio2/lio_odom</code>
action 维度不一致	<code>enable_*</code> 或 <code>base_action_mode</code> 不一致	使用同一组参数
升降动作全 0	<code>/velocity_controller/commands</code> 未发布	先启动 VR 升降节点
图像丢帧	USB/网络带宽不足	降低 fps 或减少相机
推理时底盘不动	默认发布 <code>/vla/base_path</code>	确认 path 跟踪节点订阅该话题

5.9 训练流程（lerobot-train）

策略	数据规模	适用阶段	技术特点
ACT	50~200 episodes	首个全身 baseline	预测未来 action chunk，训练快
Diffusion Policy	100~500 episodes	多解抓取和对位	条件扩散去噪，多模式动作
SmolVLA / OpenVLA / pi0	500+ episodes	语言条件泛化	视觉-语言-动作统一建模

ACT 示例：

```
lerobot-train \
  --policy.type=act \
  --dataset.repo_id=YOUR_NAME/openflex_walk_and_grasp \
  --output_dir=outputs/train/act_openflex_wholebody \
  --job_name=act_openflex_wholebody \
  --policy.device=cuda \
  --batch_size=8 \
  --steps=100000 \
  --save_freq=5000 \
  --eval_freq=0 \
  --log_freq=200
```

训练时不手工写死输入输出维度，LeRobot 会从 dataset metadata 推断。如果切换 `base_action_mode`，必须重新采集或转换数据集。

5.10 推理与部署

推理前必须关闭 VR 遥操作，保留整机 bringup、FAST-LIO2 odom 和相机。Follower 设置 `skip_send_action=False`。

```
from lerobot.policies.factory import make_policy
from lerobot_robot_openflex_follower_ros2.robot_bridge import OpenFlexFollowerRos2
from lerobot_robot_openflex_follower_ros2.config import (
    OpenFlexFollowerRos2Config, OpenFlexRos2InterfaceConfig,
)

robot = OpenFlexFollowerRos2(OpenFlexFollowerRos2Config(
    skip_send_action=False,
    relative_odom=True,
    ros2=OpenFlexRos2InterfaceConfig(
        enable_base=True,
        enable_head=True,
        enable_lift=True,
        odom_topic="/fastlio2/lio_odom",
        base_action_mode="pose",
        base_path_coordinate_mode="episode_local",
    ),
))
robot.connect()
policy = make_policy(...)

while True:
    obs = robot.get_observation()
    obs["task"] = "Walk to the table and pick up the cup"
    action = policy.select_action(obs)
    robot.send_action(action)
```

5.11 长程任务分层与渐进启用

推荐分层架构：

语言任务

-> 高层任务规划：选择目标区域和子任务

-> Nav2/NMPC：长程导航、避障、全局路径

-> VLA：到位后的底盘微调、升降、头部观察、双臂操作

-> ros2_control：关节、电机、舵轮底层闭环

渐进式启用：

阶段	功能	通过标准
Phase 0	仅双臂	5~10 条 episode 可训练并 replay
Phase 1	+头部	头部 action/obs 维度稳定
Phase 2	+升降	升降速度标签与实际运动一致
Phase 3	+底盘 pose	episode-local odom 归零正确
Phase 4	trajectory 或 Nav2+VLA	短程路径可复现

附录 核心算法技术细节与论文依据

本附录按"项目中实际使用的算法模块"整理技术细节，给出项目实现位置、核心数学模型、工程策略和参考文献/标准。

A.1 算法索引与项目实现位置

算法/机制	用途	当前实现位置	参考
四转四驱舵轮 IK/FK	/cmd_vel 到四个转向角和轮速	src/openflex_chassis/motion_control_layer/swerve_controller/src/swerve_drive_kinematics.cpp	Muir, Neuman, 1987
二阶中点法里程计	由 FK 速度积分 odom	src/openflex_chassis/motion_control_layer/swerve_controller/src/swerve_drive_odometry.cpp	数值标准
FAST-LIO2 / IESKF	MID-360 点云 + IMU 高频 LIO	src/openflex_chassis/mapping_localization_layer/fastlio2/src/lio_node.cpp	Xu et al., 2022
FAST-LIVO / FAST-LIVO2	LiDAR-IMU-Visual 融合	src/openflex_chassis/mapping_localization_layer/fast_livo/	Zherov et al., 2022

算法/机制	用途	当前实现位置	参考
ScanContext	回环检测、全局重定位	PGO/ICP 包内 ScanContext 数据库	Kim 2018
NDT	ScanContext 后的粗配准	icp_registration 配准链路	Bibe Straß 2003
ICP	PCD 地图精配准	src/openflex_chassis/mapping_localization_layer/icp_registration/src/icp_registration.cpp	Besl McK 1992
PGO / iSAM2	回环约束位姿图优化	src/openflex_chassis/mapping_localization_layer/pgo/src/pgo_node.cpp	Kaess al. 2012
HBA	离线地图精修	src/openflex_chassis/mapping_localization_layer/hba/src/hba_node.cpp	Liu et al. 2023
PCD 到 2D 栅格	3D 地图切片、滤波、膨胀	src/openflex_chassis/mapping_localization_layer/pgo/src/pcd_to_nav2_map.cpp	Moravec Elferink 2018
SmacPlanner2D / A*	Nav2 全局路径规划	src/openflex_chassis/navigation_layer/swerve_navigation/config/nav2_params.yaml	Hartmann 1968
NMPC / acados	全向底盘局部路径跟踪	src/openflex_chassis/navigation_layer/nmpc_controller/	Mayhew al. 2022 acados 2022
VR 双臂 IK	Pico 6DoF → 双臂关节	src/openflex_armx/openarmx_teleop_vr/	Pino DLS
CANopen/CiA402	升降台驱动	src/openflex_lift_slide/lift_slide_driver/	CiA 3 CiA 4
LeRobot/ACT/VLA	全身数据采集与推理	external openflex_vla package/	ACT; Diffusion Policymaker Ope pi0

A.2 四转四驱舵轮运动学与里程计

项目实现：

- [src/openflex_chassis/motion_control_layer/swerve_controller/src/swerve_drive_kinematics.cpp](#)
- [src/openflex_chassis/motion_control_layer/swerve_controller/src/swerve_drive_controller.cpp](#)

设底盘速度为车体系 $v = [vx, vy, \omega]^T$ ，第 i 个舵轮模块安装坐标 (x_i, y_i) 。刚体平面运动给出模块接地点速度：

```
vx_i = vx - omega * y_i
vy_i = vy + omega * x_i
```

逆运动学矩阵：

```
[vx_i]   [1  0  -y_i] [vx  ]
[vy_i] = [0  1   x_i] [vy  ]
              [omega]
```

模块目标轮速和转向角：

```
speed_i = sqrt(vx_i^2 + vy_i^2)
angle_i = atan2(vy_i, vx_i)
```

当前项目参数：

模块	(x_i, y_i) m	转向限位	轮半径
FL	$(0.21, 0.26)$	$[-1.5708, 1.5708]$ rad	0.075 m
FR	$(0.21, -0.26)$	$[-1.5708, 1.5708]$ rad	0.075 m

模块	(x_i, y_i) m	转向限位	轮半径
BL	(-0.21, 0.26)	[-1.5708, 1.5708] rad	0.075 m
BR	(-0.21, -0.26)	[-1.5708, 1.5708] rad	0.075 m

正运动学使用 Moore-Penrose 伪逆：

```
v_chassis = (A^T A)^-1 A^T b
```

转向优化：比较正向/反向两个候选，选择代价小者：

```
正向: angle = target, speed = +speed
反向: angle = target + pi, speed = -speed
cost = abs(delta_angle) + 2.0 * clamp_error
```

里程计积分：使用二阶中点法：

```
delta_heading = omega * dt
mid_heading = heading + delta_heading / 2
x += (vx*cos(mid_heading) - vy*sin(mid_heading)) * dt
y += (vx*sin(mid_heading) + vy*cos(mid_heading)) * dt
heading += delta_heading
```

参考论文：

- Muir, P. F., Neuman, C. P. "Kinematic Modeling for Feedback Control of an Omnidirectional Wheeled Mobile Robot." ICRA, 1987.
- Campion, G., Bastin, G., D'Andrea-Novel, B. "Structural Properties and Classification of Kinematic and Dynamic Models of Wheeled Mobile Robots." IEEE T-RA, 1996.

A.3 FAST-LIO2 与 ikd-Tree

项目实施：

- src/openflex_chassis/mapping_localization_layer/fastlio2/src/lio_node.cpp

状态：

```
x = {R_wi, p_wi, v_wi, b_g, b_a, g, R_il, p_il}
```

处理流程：

- IMU 初始化：估计重力方向和陀螺零偏
- IMU 传播：角速度/加速度积分状态与协方差
- 点云去畸变：按每个点的相对时间补偿
- 最近邻平面拟合：在 ikd-Tree 中找近邻
- IESKF 更新：最小化点到平面残差
- 增量地图维护：新点插入 ikd-Tree

点到平面残差：

```
r_j = n_j^T (R_wl * p_lj + p_wl) + d_j
```

IESKF 迭代：

```
delta_x = K * (z - h(x))
x <- x boxplus delta_x
```

本项目工程改动：

- 导航模式把输出补偿到 base_link
- 对 map -> odom 加平面约束，只保留 x/y/yaw
- 静止时可冻结 map -> odom 更新

参考论文：

- Xu, W. et al. "FAST-LIO2: Fast Direct LiDAR-Inertial Odometry." IEEE T-RO, 2022.
- Cai, Y. et al. "ikd-Tree: An Incremental KD Tree for Robotic Applications." arXiv, 2021.

A.4 FAST-LIVO2 视觉-激光-惯性融合

项目实施：

- `src/openflex_chassis/mapping_localization_layer/fast_livo/src/LIVMapper.cpp`
 - `src/openflex_chassis/mapping_localization_layer/fast_livo/src/vio.cpp`

FAST-LIVO/FAST-LIVO2 在 LIO 前端基础上加入相机视觉约束，用于几何退化但纹理明显的场景。

输入：

```
MID-360 point cloud + IMU + RealSense/D435 image
```

融合思想：

LiDAR/IMU:

- IMU 传播给出连续运动先验
- LiDAR 点到平面残差约束尺度、姿态和平移

Vision:

- 图像特征或直接法光度误差提供视觉观测
- 视觉点与体素地图关联形成稀疏约束

联合更新:

- 在同一状态估计框架中融合几何残差和视觉残差
- 输出 LIO/LIVO 兼容里程计

参考论文：

- Zheng, C. et al. "FAST-LIVO: Fast and Tightly-coupled Sparse-Direct LiDAR-Inertial-Visual Odometry." IROS/RA-L, 2022.
- Zheng, C. et al. "FAST-LIVO2: Fast, Direct LiDAR-Inertial-Visual Odometry." arXiv, 2024.

A.5 ScanContext、NDT、ICP 与重定位

项目实施：

- `src/openflex_chassis/mapping_localization_layer/icp_registration/src/icp_registration.cpp`

定位链路目标：

```
map_T_odom = map_T_laser * inverse(odom_T_laser)
```

ScanContext：将点云转换为极坐标描述子：

```
ring = floor(radius / max_radius * N_ring)
sector = floor(theta / 2pi * N_sector)
SC[ring, sector] = max_z_in_bin
```

匹配时用 RingKey 做 KD-tree 候选检索，再用 SectorKey 估计 yaw 偏移。

NDT：把目标点云划成栅格，每格估计高斯分布 $N(\mu, \sigma)$ ，源点变换后落入目标格的概率形成优化目标。

ICP：迭代最近邻匹配和刚体变换估计：

```
given T_k:
q_i = nearest_neighbor(T_k p_i, target_map)
T_{k+1} = argmin_T sum_i || T p_i - q_i ||^2
```

三级定位策略：

1. ScanContext -> NDT -> ICP
 2. 若 NDT 失败：ScanContext 初值 -> ICP
 3. 若 ScanContext 不可用：XY/Yaw 网格多假设 -> ICP

参考文献：

- Kim, G., Kim, A. "Scan Context: Egocentric Spatial Descriptor for Place Recognition." IROS, 2018.
- Biber, P., Strasser, W. "The Normal Distributions Transform." IROS, 2003.
- Besl, P. J., McKay, N. D. "A Method for Registration of 3-D Shapes." IEEE TPAMI, 1992.
- Chen, Y., Medioni, G. "Object Modelling by Registration of Multiple Range Images." Image and Vision Computing, 1992.

A.6 PGO、iSAM2 与 HBA 地图精修

项目实施：

- `src/openflex_chassis/mapping_localization_layer/pgo/src/pgo_node.cpp`
- `src/openflex_chassis/mapping_localization_layer/hba/src/hba_node.cpp`

PGO 把 FAST-LIO2 轨迹离散成关键帧图：

```
nodes: X_0, X_1, ..., X_N
odometry factors: Z_i,i+1 = inverse(X_i) * X_i+1
loop factors:      Z_i,j    = ScanContext + NDT/ICP 得到的相对位姿
```

优化目标：

```
argmin_X ||r_prior||^2
+ sum_i || Log( inverse(Z_i,i+1) * inverse(X_i) * X_i+1 ) ||^2_0omega_odom
+ sum_(i,j) || Log( inverse(Z_i,j) * inverse(X_i) * X_j ) ||^2_0omega_loop
```

iSAM2 使用 Bayes Tree 做增量平滑与映射。PGO 不直接改 FAST-LIO2 内部状态，而是发布 `/pgo/offset`。

HBA（Hierarchical Bundle Adjustment）用于离线地图精修：

原始关键帧 -> 局部窗口 BA -> 局部约束 -> 全局 LM 优化 -> 重新拼接地图

参考文献：

- Kaess, M. et al. "iSAM2: Incremental Smoothing and Mapping Using the Bayes Tree." IJRR, 2012.
- Dellaert, F. "Factor Graphs and GTSAM." Technical Report, 2012.
- Liu, Xiyuan et al. "Large-Scale LiDAR Consistent Mapping Using Hierarchical LiDAR Bundle Adjustment." IEEE RA-L, 2023.

A.7 3D 点云到 Nav2 2D 栅格/扫描

项目实施：

- 离线地图：`src/openflex_chassis/mapping_localization_layer/pgo/src/pcd_to_nav2_map.cpp`
- 实时扫描：`src/openflex_chassis/navigation_layer/pointcloud_to_laserscan/src/pointcloud_to_laserscan_node.cpp`

离线 3D PCD 到 2D 栅格地图流程：

1. 按 z 范围切片
observed_hits → FREE
obstacle_hits → OCCUPIED
ceiling_hits (无 obstacle) → FREE
2. 噪声处理
SOR 统计离群点移除
BFS 连通域去除小障碍簇
3. Raycasting
Bresenham 直线标记 FREE
4. Inflation
对 OCCUPIED 栅格按半径膨胀

实时 `PointCloud2` -> `LaserScan`：

1. 点云变换到 `target_frame`
2. 按 `min_height/max_height` 过滤
3. 按 `angle_increment` 分桶
4. 每桶保留最近距离
5. 发布 `/scan`

参考文献：

- Moravec, H., Elfes, A. "High Resolution Maps from Wide Angle Sonar." ICRA, 1985.
- Bresenham, J. E. "Algorithm for Computer Control of a Digital Plotter." IBM Systems Journal, 1965.

A.8 Nav2 SmacPlanner2D、局部 A* 与 NMPC

项目实施：

- `src/openflex_chassis/navigation_layer/swerve_navigation/config/nav2_params.yaml`
- `src/openflex_chassis/navigation_layer/nmpc_controller/src/nmpc_controller.cpp`
- `src/openflex_chassis/navigation_layer/nmpc_controller/script/generate_solver.py`

全局规划 A：*

$$f(n) = g(n) + h(n)$$

局部 A（NMPC 内部补丁）：

$$\text{neighbor cost} = \text{step_distance} * (1 + 5 * \max(0, \text{normalized_cost} - \text{inflation_cost}))$$

路径平滑：

$$p_i \leftarrow p_i + w_{\text{data}} * (p_{\text{i_original}} - p_i) + w_{\text{smooth}} * (p_{\{i-1\}} + p_{\{i+1\}} - 2p_i)$$

NMPC 模型：

$$\begin{aligned} x &= [px, py, theta, vx, vy, omega] \\ u &= [ax, ay, alpha] \end{aligned}$$

有限时域优化：

$$\begin{aligned} \text{minimize} \quad & \sum_{k=0}^{N-1} ||x_k - x_{\text{ref_k}}||_Q^2 \\ & + \sum_{k=0}^{N-1} ||u_k||_R^2 \\ & + \sum_{k=0}^{N-1} w_c * \text{costmap}(x_{\text{ref_k}})^2 \\ & + ||x_N - x_{\text{ref_N}}||_{Q_e}^2 \\ \text{subject to:} \quad & x_{\{k+1\}} = f(x_k, u_k) \\ & \text{速度和加速度约束} \end{aligned}$$

acados 采用 SQP_RTI，每个控制周期只执行一次实时迭代。

参考文献：

- Hart, P. E., Nilsson, N. J., Raphael, B. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." IEEE TSSC, 1968.
- Mayne, D. Q. et al. "Constrained Model Predictive Control: Stability and Optimality." Automatica, 2000.
- Verschueren, R. et al. "acados: A Modular Open-Source Framework for Fast Embedded Optimal Control." Mathematical Programming Computation, 2022.

A.9 VR 遥操作、双臂 IK 与安全限幅

项目实施：

- UDP 桥接：`src/openflex_vr_bridge/src/pose_bridge_node.cpp`
- 双臂 VR：`src/openflex_armx/openarmx_teleop_vr/openarmx_teleop_vr/openarmx_teleop_vr_node.py`
- 头部 VR：`src/openflex_head/openarmx_head_teleop_vr_pico/openarmx_head_teleop_vr_pico/head_teleop_node.py`
- 底盘摇杆：`src/openflex_chassis/system_bringup_layer/swerve_bringup/scripts/vr_teleop_node.py`

UDP 协议格式：

$$\begin{aligned} \text{HAND} \quad & \text{<L/R> px py pz qx qy qz qw trigger grip rate timestamp_ns} \\ \text{BTN} \quad & \text{<L/R> <A/B/X/Y/J> pressed timestamp_ns} \end{aligned}$$

```
JOY  <L/R> x y timestamp_ns
HEAD/WAIST px py pz qx qy qw timestamp_ns
```

双臂 IK（阻尼最小二乘）：

```
delta_q = J^T (J J^T + lambda^2 I)^-1 e
q <- q + delta_q
```

头部相对姿态控制：

```
q_rel = inverse(q_ref) * q_current
(rel_yaw, rel_pitch) = euler_yxz(q_rel)
target = anchor + scale * relative_angle
```

底盘摇杆：死区和指数曲线：

```
deadzone(v) = 0, if |v| < dz
              = sign(v) * (|v|-dz)/(1-dz)
expo(v) = sign(v) * |v|^gamma
```

参考文献：

- Carpentier, J. et al. "The Pinocchio C++ Library." SII, 2019.
- Nakamura, Y., Hanafusa, H. "Inverse Kinematic Solutions with Singularity Robustness." ASME JDSMC, 1986.

A.10 CANopen/CiA402 升降台控制与回零

项目实施：

- `src/openflex_lift_slide/lift_slide_driver/src/lift_slide_hardware_interface.cpp`

关键对象字典：

对象	含义
0x6040	Controlword
0x6041	Statusword
0x6060/0x6061	Mode of Operation / Display
0x6064	Position Actual Value
0x606C	Velocity Actual Value
0x607A	Target Position
0x60FF	Target Velocity
0x60FD	Digital Inputs
0x6098/0x6099/0x609A	Homing method/speeds/acceleration

单位换算：

```
position_m = encoder_counts / counts_per_meter
counts_per_meter = 2,000,000
```

CiA402 上电启用序列：

```
NMT start
Fault reset if needed
Shutdown      controlword = 0x0006
Switch on     controlword = 0x0007
Enable op     controlword = 0x000F
Set mode      0x6060 = velocity/position/homing
```

软件回零逻辑：


```
1. 配置 HOME/POT/NOT 数字输入映射
2. 暂停异步 SDO 反馈线程
3. 读取 0x6064/0x606C/0x6041/0x60FD
4. 若 HOME 已触发, 当前位置作为零点
5. 否则先向上搜索 HOME
6. 若先碰到上限 NOT, 则反向向下搜索 HOME
7. HOME 连续两次采样有效后停止
8. position_offset_m = physical_position_m
9. 用户坐标 hw_position = physical_position - position_offset
10. 保存 ~/.lift_slide_calibration.yaml
```

参考标准:

- CiA 301: CANopen Application Layer and Communication Profile.
- CiA 402: CANopen Device Profile for Drives and Motion Control.

A.11 LeRobot VLA 策略、ACT、Diffusion Policy 与 VLA 模型

项目实施:

- Follower: [external openflex_vla package/lerobot_robot_openflex_follower_ros2/](#)
- Teleoperator: [external openflex_vla package/lerobot_teleoperator_openflex_leader_ros2/](#)
- Launch: [external openflex_vla package: launch/wholebody_vla_record.launch.py](#)

底盘 [pose/trajectory](#) 标签积分公式 (平面刚体积分) :

```
theta_{t+1} = theta_t + omega * dt
x_{t+1} = x_t + (cos(theta_t) * vx - sin(theta_t) * vy) * dt
y_{t+1} = y_t + (sin(theta_t) * vx + cos(theta_t) * vy) * dt
```

ACT (Action Chunking Transformer) :

```
policy(o_t) -> [a_t, a_{t+1}, ..., a_{t+H-1}]
```

训练用 L1/L2 动作重构损失和 CVAE latent 正则; 部署时用时间集成做指数加权平均。

Diffusion Policy:

```
epsilon ~ N(0, I)
a_k = sqrt(alpha_k) * a_0 + sqrt(1-alpha_k) * epsilon
network predicts epsilon conditioned on observations
```

推理时从噪声动作序列迭代去噪, 能表达多模态动作分布。

参考文献:

- Zhao, T. Z. et al. "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware." RSS, 2023. (ACT/ALOHA)
- Chi, C. et al. "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion." RSS, 2023.
- Brohan, A. et al. "RT-1: Robotics Transformer for Real-World Control at Scale." RSS, 2023.
- Kim, M. J. et al. "OpenVLA: An Open-Source Vision-Language-Action Model." arXiv, 2024.
- Black, K. et al. "pi0: A Vision-Language-Action Flow Model for General Robot Control." arXiv, 2024.
- Cadene, R. et al. "LeRobot: State-of-the-art Machine Learning for Real-World Robotics in Pytorch." Hugging Face, 2024.

A.12 参考论文与标准清单

方向	引用
舵轮/全向底盘	Muir & Neuman, 1987; Campion et al., 1996
LiDAR-Inertial Odometry	Xu et al., "FAST-LIO2", IEEE T-RO 2022; Cai et al., "ikd-Tree", 2021
LiDAR-Visual-Inertial Odometry	Zheng et al., "FAST-LIVO", 2022; "FAST-LIVO2", 2024
回环/重定位	Kim & Kim, "Scan Context", IROS 2018; "Scan Context++", IEEE T-RO 2021
点云配准	Besl & McKay, TPAMI 1992; Chen & Medioni, 1992; Biber & Strasser, "NDT", IROS 2003
图优化/地图精修	Kaess et al., "iSAM2", IJRR 2012; Dellaert, "GTSAM", 2012; Liu et al., "HBA", RA-L 2023
栅格地图	Moravec & Elfes, ICRA 1985; Bresenham, IBM Systems Journal 1965

方向	引用
路径规划	Dijkstra, 1959; Hart, Nilsson & Raphael, A*, 1968
最优控制/NMPC	Mayne et al., Automatica 2000; Verschueren et al., acados, 2022
IK/机器人动力学	Nakamura & Hanafusa, 1986; Carpentier et al., Pinocchio, 2019
CANopen/驱动	CiA 301; CiA 402
VLA/模仿学习	ACT/ALOHA, 2023; Diffusion Policy, 2023; RT-1/RT-2, 2023; OpenVLA, 2024; pi0, 2024; LeRobot, 2024
移动操作/底盘 VLA 数据	Galaxea Open-World/G0; AgiBot World; Mobile ALOHA; NaVILA

文档结束。 本高级篇涵盖传感器系统（第一章）、建图算法与流程（第二章）、三级定位策略与 TF 架构（第三章）、Nav2 导航与 NMPC 控制器（第四章）、全身 VLA 数据采集训练推理（第五章）以及核心算法论文索引（附录）。配合基础篇使用，可完整理解 OpenFlex 全系统的工程实现和算法原理。